Python Exception Handling Practice Problems

HackerRank Style Practice Set Covering `try`, `except`, `else`, `finally`, `raise`, and `assert`.

TRY-EXCEPT-ELSE-FINALLY | RAISE & CUSTOM EXCEPTIONS | ASSERT PRE-CONDITIONS | EXCEPTION HIERARCHY

Task 1: Safe Division Calculator

ZeroDivisionError & ValueError

PROBLEM STATEMENT

You are given two inputs, a and b, as strings representing numbers. Perform division `a / b` and output the integer division result. Handle non-numeric inputs and division by zero cleanly.

EXCEPTION HANDLING REQUIREMENTS

Catch `ValueError` when inputs cannot be converted to integers and print `"Error Code: ValueError"`. Catch `ZeroDivisionError` when division by zero occurs and print `"Error Code: ZeroDivisionError"`.

INPUT FORMAT

```
Two lines, each containing a string input representing a or b.
```

OUTPUT FORMAT

```
Print integer division result or designated error code message.
```

SAMPLE INPUT

```
10 0
```

SAMPLE OUTPUT

```
Error Code: ZeroDivisionError
```

Task 2: List Index & Type Safety Guard

IndexError & TypeError

PROBLEM STATEMENT

Given a predefined list of elements `items = [10, 20, 30, 40, 50]`, read a target index and a divisor from standard input. Retrieve `items[index]` and divide it by the divisor.

EXCEPTION HANDLING REQUIREMENTS

Use a single `try` block with separate `except` blocks: `IndexError` → `"Index out of range"`, `ValueError` → `"Invalid integer input"`, `ZeroDivisionError` → `"Cannot divide by zero"`.

INPUT FORMAT

```
Line 1: Index integer Line 2: Divisor integer
```

OUTPUT FORMAT

```
Print division result rounded to 2 decimals or error message.
```

SAMPLE INPUT

```
7 2
```

SAMPLE OUTPUT

```
Index out of range
```

Task 3: Bank Account Withdrawal Validator

raise & Custom Exception

PROBLEM STATEMENT

Implement a withdrawal mechanism for a bank account starting with a given balance. Read the withdrawal amount and evaluate whether it exceeds the current balance or is non-positive.

EXCEPTION HANDLING REQUIREMENTS

Define a custom exception `InsufficientFundsError(Exception)`. Raise `InsufficientFundsError` if withdrawal > balance. Raise `ValueError` if withdrawal <= 0. Handle both and print appropriate error messages.

INPUT FORMAT

```
Line 1: Initial balance (float) Line 2: Withdrawal amount (float)
```

OUTPUT FORMAT

```
Print new balance or custom exception message.
```

SAMPLE INPUT

```
500.00 650.00
```

SAMPLE OUTPUT

```
InsufficientFundsError: Withdrawal amount exceeds balance
```

Task 4: Age Assertion Check

assert Statements

PROBLEM STATEMENT

Write a voting eligibility verification function. Read an integer age and use an `assert` statement to validate that age is within a valid human lifespan (0 to 120 inclusive) before checking eligibility (age >= 18).

EXCEPTION HANDLING REQUIREMENTS

Use `assert 0 <= age <= 120, "Invalid age provided"`. Catch the resulting `AssertionError` in a `try-except` block and print the exact assertion message.

INPUT FORMAT

```
A single integer representing age.
```

OUTPUT FORMAT

```
Print "Eligible", "Ineligible", or assertion error message.
```

SAMPLE INPUT 1

```
-5
```

SAMPLE OUTPUT 1

```
AssertionError: Invalid age provided
```

Task 5: Complete Flow Control (try-except-else-finally)

else & finally Blocks

PROBLEM STATEMENT

Parse a string input into an integer representing temperature in Celsius and convert it to Fahrenheit (`F = (C * 9/5) + 32`). Demonstrate complete execution flow using all four blocks: `try`, `except`, `else`, and `finally`.

EXCEPTION HANDLING REQUIREMENTS

- `try`: Parse input integer.
- `except ValueError`: Print `"Conversion Failed"`.
- `else`: Calculate Fahrenheit and print `"Converted: [F]F"`.
- `finally`: Always print `"Execution Complete"`.

INPUT FORMAT

```
A single string input.
```

OUTPUT FORMAT

```
Output lines corresponding to triggered flow blocks.
```

SAMPLE INPUT

```
25
```

SAMPLE OUTPUT

```
Converted: 77.0F Execution Complete
```

Task 6: Multiple Exception Grouping & Dictionary Lookup

KeyError & TypeError

PROBLEM STATEMENT

Given a predefined dictionary `scores = {"alice": 90, "bob": 85, "charlie": "N/A"}`, take a student name input, look up their score, and add 5 bonus points to it.

EXCEPTION HANDLING REQUIREMENTS

Catch `(KeyError, TypeError)` in a combined `except` clause when the key is missing or the score value cannot be added to an integer. Print `"Invalid Student or Score Data"`.

INPUT FORMAT

```
Single string containing student name.
```

OUTPUT FORMAT

```
Print updated score or combined exception message.
```

SAMPLE INPUT

```
charlie
```

SAMPLE OUTPUT

```
Invalid Student or Score Data
```

Task 7: Resource Cleanup Manager

finally Guarantee

PROBLEM STATEMENT

Simulate a database connection object with methods `connect()`, `execute(query)`, and `close()`. Execute a user-provided SQL-like query string. If query contains `"DROP"`, raise a `PermissionError`.

EXCEPTION HANDLING REQUIREMENTS

Ensure `close()` is guaranteed to run inside a `finally` block regardless of whether `execute()` succeeds or raises an exception. Print `"Connection Closed"` upon closing.

INPUT FORMAT

```
A single string representing query.
```

OUTPUT FORMAT

```
Print execution status/error, followed by cleanup confirmation.
```

SAMPLE INPUT

```
DROP TABLE users
```

SAMPLE OUTPUT

```
Error: Permission Denied Connection Closed
```

Task 8: Password Strength Assertor

assert & raise Chain

PROBLEM STATEMENT

Create a registration password validator function `validate_password(password)` that enforces three structural rules:

1. Length must be at least 8 characters.
2. Must contain at least one digit.
3. Must contain at least one uppercase letter.

EXCEPTION HANDLING REQUIREMENTS

Use `assert` statements for each rule with specific error messages (e.g., `"Too short"`, `"Missing digit"`). Catch `AssertionError` and re-raise it as a custom `WeakPasswordError`.

INPUT FORMAT

```
Single string containing password.
```

OUTPUT FORMAT

```
Print "Password Valid" or custom error details.
```

SAMPLE INPUT

```
hello123
```

SAMPLE OUTPUT

```
WeakPasswordError: Missing uppercase letter
```

Task 9: Nested Exception Handler & Re-raising

Nested try & raise

PROBLEM STATEMENT

Process a list of numerical strings. In an inner `try` block, convert string to float and compute square root. If number is negative, catch `ValueError` and re-raise a `MathDomainError` to outer block.

EXCEPTION HANDLING REQUIREMENTS

Inner block catches conversion errors. Inner block raises `MathDomainError` on negative inputs. Outer `try-except` catches `MathDomainError` and prints `"Domain Error: Negative value encounter"`.

INPUT FORMAT

```
Space-separated line of numbers.
```

OUTPUT FORMAT

```
Print processed roots until exception is caught.
```

SAMPLE INPUT

```
16 -9 25
```

SAMPLE OUTPUT

```
4.0 Domain Error: Negative value encounter
```

Task 10: Multi-Stage Order Processing Pipeline

Full Flow & Custom Hierarchy

PROBLEM STATEMENT

Implement an e-commerce checkout handler `process_order(item_id, quantity, price, discount)` that evaluates inputs across multiple stages:

1. Pre-condition check using `assert quantity > 0 and price > 0`.
2. Discount check: raise `ValueError` if `discount < 0 or discount > 100`.
3. Calculate total = `(price * quantity) * (1 - discount/100)`.

EXCEPTION HANDLING REQUIREMENTS

Wrap processing in `try-except-else-finally`:

- Catch `AssertionError` → Print `"Invalid Order Quantity or Price"`.
- Catch `ValueError` → Print `"Invalid Discount Percentage"`.
- `else` → Print `"Order Processed Total: $[total]"`.
- `finally` → Print `"Audit Log Saved"`.

INPUT FORMAT

```
Line 1: Item ID (str) Line 2: Quantity (int) Line 3: Price (float) Line 4: Discount % (float)
```

OUTPUT FORMAT

```
Print pipeline output and audit log line.
```

SAMPLE INPUT

```
ITEM101 2 50.0 150.0
```

SAMPLE OUTPUT

```
Invalid Discount Percentage Audit Log Saved
```

Python Exception Handling Reference Guide

- **`try` block:** Wraps code that may raise an exception during execution.
- **`except` block:** Catches and handles specific exceptions. Multiple exceptions can be grouped as a tuple: `except (TypeError, ValueError):`.
- **`else` block:** Executes ONLY if the `try` block completes successfully without raising any exceptions.
- **`finally` block:** ALWAYS executes before leaving the `try` structure, making it ideal for cleanup actions (e.g., closing files/connections).
- **`raise` statement:** Manually triggers an exception. Can be used to re-raise current exceptions or throw custom exception instances.
- **`assert` statement:** Tests a condition (`assert condition, "Error Message"`). Raises `AssertionError` if condition evaluates to `False`.